SEAL_directed + node attributes | Detailed workflow

Repository-grounded data flow, directional encoding and DGCNN training configuration

DATA & SAMPLING

1 Load attributed MAT

net: directed binary adjacency
group: 44-column node matrix
role: consumer/resource/both

2 Split positive links

Keep ordered non-self links (i, j)
Shuffle with run-specific seed
ceil(0.90 × L) train; rest test

3 Sample directed negatives

Unknown ordered resource → consumer pairs
Uniform without replacement; 1:1 balance
Fallback to all directed non-links if needed

4 Build observed graph A

Copy binary net
Remove held-out positive i → j links
Preserve reverse j → i context if observed

each sampled ordered pair

SUBGRAPH & FEATURES

5 Candidate pair (source, target)

Resource endpoint stays first
Consumer endpoint stays second
Graph label: interaction = 1, non-link = 0

6 Extract h = 1 neighborhood

Collect incoming + outgoing neighbors
Induce directed local adjacency
Mask source → target inside every subgraph

7 Directional structural tags

Weak-projection DRNL for other nodes
source label = 1; target label = 2
Shift remaining positive labels by +2

8 Attach 44 node attributes

2 mass features · 6 taxonomy levels
10 metabolic · 7 movement · 19 life-stage
Slice rows to the extracted subgraph

encoded graph examples

LEARNING & EVALUATION

9 Create DGCNN input

One-hot structural tag ⊕ 44 attributes
Weak-undirected NetworkX projection
No Node2Vec; no edge features

10 Learn graph representation

Latent widths: 32, 32, 32, 1
Concatenate convolution outputs
SortPooling: 60th percentile, min k = 10

11 Optimise classifier

1D convolutions + MLP(128), dropout
Adam, lr = 10^{-4}; batch size 50
50 epochs; select minimum validation loss

12 Evaluate held-out pairs

Class-1 probability is link score
Positive if score > 0.5
Report seven predictive metrics per run

IMPLEMENTATION BOUNDARY Direction affects ordered sampling, target masking and endpoint labels; the current DGCNN aggregates on a weak-undirected projection.

The role vector constrains negative candidates; it is not concatenated to the learned 44-feature node representation. No link weights are used.

Sources: Main_directed.py · util_functions_directed.py · build_node_attribute_mats.py · pytorch_DGCNN